Lliurament pdf

BLOC A. CONCEPTES BÀSICS (Escalfament)

Exercici A1
Escriu amb MÀXIM 1 línia cadascun:

a) API

b) HTTP

c) REST

d) Recurs

e) Endpoint

Exercici A2

Uneix cada verb HTTP amb la descripció (posa la lletra):

1. GET 2. POST 3. PUT 4. PATCH 5. DELETE

a) Eliminar un recurs
b) Crear un recurs nou
c) Llegir dades
d) Actualitzar parcialment
e) Substituir totalment

Exercici A3

Quina opció és més “REST” per crear un producte?

a) GET /crearProducte?nom=A
b) POST /productes
c) POST /crearProducte
d) PUT /productes/crear

Exercici A4 (Veritat / Fals – 4’)

- Cada petició HTTP inclou sempre un body.
- Un 404 vol dir que el servidor està caigut.
- REST obliga a fer servir JSON.
- Un 201 indica que s’ha creat un recurs nou.
- PATCH i PUT són exactament iguals.

BLOC B. DISSENY D'ENDPOINTS

Context: Sistema Biblioteca amb recursos: llibres, usuaris, préstecs.

Exercici B1

Defineix MÈTODE + PATH per:

- Llistar tots els llibres
- Afegir un llibre nou
- Consultar dades del llibre amb id 90
- Eliminar el llibre 77
- Actualitzar només el títol del llibre 12
- Substituir completament dades del llibre 12

Exercici B2

Escriu un possible body JSON per crear un llibre (mínim 3 camps raonables).

Exercici B3

Quin codi d'estat retornaries per cada situació:

- Llista de llibres obtinguda correctament
- Llibre creat
- Llibre 123 no existeix
- JSON mal format
- Error inesperat de servidor

BLOC C. ANALITZAR I CORREGIR
Exercici C1 (Detecta 5 errors)

Petició proposada:

```
POST /crearLlibre/ HTTP/1.1
Host: api.biblio.local
ContentType: text/plain
Body:
{ titol: "DDD", "autor": "Evans" }
```

Quins errors hi veus? Escriu-los i proposa correcció.

Exercici C2 (Resposta qüestionable)

Resposta rebuda:

```
HTTP/1.1 200 OK
Content-Type: application/json
{ "error": "Llibre no trobat" }
```

Quin codi d'estat hauria de ser? Per què?

Exercici C3 (Req vs Resp – 5')

Marca quin camp pertany a Request (R), quin a Response (S):

```
Status Code
Body amb JSON enviat pel client
Authorization Header
Content-Type: application/json (del que torna el servidor)
Accept: application/json
```

BLOC D. SEQÜÈNCIES DE PETICIONS
Exercici D1 (Orde lògic – 6')

Ordena (1→5) les accions per signar-se i veure perfil:

__ GET /usuaris/jo (requereix estar autenticat)

__ POST /auth/login (rep token)

__ Guardar el token localment

__ GET /auth/status (opcional, comprova token)

__ Fer fetch amb Header Authorization: Bearer <token>

Exercici D2 (Pensar l'estat – 5')

Per què el servidor "no recorda" que ja hem fet login si no enviem el token? Explica-ho en 2 frases (paraula clau: stateless).

BLOC E. CURL / FETCH
Exercici E1 (curl)

Escriu un curl per: GET https://api.exemple.com/usuaris acceptant JSON.

Exercici E2 (curl amb body)

Crear usuari amb email “ana@test.com” i nom “Ana”: endpoint POST /usuaris a https://api.exemple.com. Afegeix header correcte.

Exercici E3 (fetch)

Escriu codi JavaScript (fetch) per fer GET /llibres i mostrar per consola el nombre total (suposa que la resposta és un array JSON).

BLOC H. REPÀS DE CODIS D'ESTAT
Exercici H1 (Classifica)

Assigna descripció a codi: 200, 201, 400, 401, 403, 404, 500

a) No trobat

b) Creat

c) Correcte

d) Error servidor

e) Falta autenticació

f) Prohibit (sense permís)

g) Petició mal construïda

Exercici H2 (Justifica)

Per què no és bona idea retornar sempre 200 i posar l'error dins del JSON? (2 raons)

BLOC G. MINI CAS D'ÚS (PROJECTE EXEMPLE)
Context: API de reserves de sales. Recursos: usuaris, sales, reserves.

Acció: Crear una reserva autenticada.

Exercici G1 (Seqüència)

Descriu (llista numerada) les peticions necessàries des de "usuari nou" fins "reserva creada". Inclou verbs i paths.

Exercici G2 (Error esperat)

Si intento reservar una sala inexistent (id 9999), quin codi d'estat i missatge curt retornaries?